



UNIVERSIDAD DE LOS ANDES
Facultad de Ingeniería y Ciencias Aplicadas
Arquitectura de Computadores 2026-20

Proyecto N°1

Grupo 16:
Andrés Berríos
Víctor Parada
Sebastián Trigo

Profesor de Cátedra: Jorge Gómez M.

9 de septiembre de 2026

Resumen

El presente proyecto tiene como objetivo el diseño e implementación de una calculadora de 4 bits, desarrollada utilizando el lenguaje *Verilog*. El diseño es programado y verificado físicamente sobre una *FPGA Lattice iCE40 HX1K* (*FPGA: Field Programmable Gate Array*), la cual se encuentra integrada en la placa de desarrollo *Nandland Go Board*. Esta placa cuenta con un conjunto reducido de recursos de entrada, específicamente cuatro botones, los cuales deberán ser utilizados como la única interfaz de interacción del usuario con la calculadora.

Se realiza el flujo completo del diseño digital, desde la etapa conceptual hasta la implementación en hardware. Se comienza con el diseño de los circuitos a partir de tablas de verdad y mapas de Karnaugh. A partir de estas expresiones, se procede a la implementación de la lógica correspondiente utilizando compuertas básicas. Posteriormente, dicha lógica es descrita formalmente mediante el lenguaje *Verilog*, lo que permite representar el comportamiento del circuito de manera estructurada y modular.

Una vez descrito el circuito, se realiza su simulación con el fin de verificar que su comportamiento sea el esperado antes de proceder a la implementación física. Para complementar esta verificación, se utiliza la herramienta *GTKWave*, la cual permite visualizar gráficamente las señales internas y externas del circuito a lo largo del tiempo, facilitando la detección de errores de diseño. Finalmente, una vez validado el funcionamiento mediante simulación, el diseño es programado sobre la *FPGA*, donde se realiza una demostración práctica de su funcionamiento utilizando los botones disponibles en la placa.

En cuanto a las operaciones aritméticas que debe soportar la calculadora, es importante destacar que todos los cálculos deben realizarse utilizando exclusivamente 4 bits. Esto implica que, en caso de que una operación genere un resultado que produzca *overflow*, únicamente se conservarán los cuatro bits menos significativos del resultado, descartando cualquier información adicional generada por dicho desbordamiento.

Finalmente, para la entrega del proyecto se crea un repositorio en *GitHub*, el cual será utilizado como medio oficial para la entrega final. Este repositorio contiene la totalidad de los archivos correspondientes al proyecto, un archivo *README.md* y este mismo informe donde se detallen los resultados obtenidos durante el desarrollo.



Índice

1. Módulos creados para la FPGA	4
1.1. sumador_1bit.v	4
1.2. full_adder_1bit.v	4
1.3. sumador_4bit.v	5
1.4. restador_4bit.v	5
1.5. restador_inverso_4bit.v	5
1.6. magnitud_4bit.v	6
1.7. display_hex_7seg.v	6
1.8. display_signo_7seg.v	6
1.9. visualizador_valor_4bit.v	7
1.10. shift_right_4bit.v	7
1.11. shift_left_4bit.v	7
1.12. leds_operacion.v	8
1.13. visualizacion_estado.v	8
1.14. selector_operacion_4bit.v	8
1.15. selector_op2_4bit.v	9
1.16. registro_operacion_3bit.v	9
1.17. registro_resultado_4bit.v	9
1.18. registro_ajustable_4bit.v	10
1.19. botones_goboard.v	10
1.20. control_valores.v	11
1.21. control_estados.v	11
1.22. control_entrada.v	11
1.23. calculadora_sistema.v	12
1.24. calculadora_core_4bit.v	12
1.25. top.v	13
2. Funcionamiento de la FPGA y Compilamiento de Código	14



3. Testbenching de la FPGA

16

1. Módulos creados para la FPGA

En este apartado, se incluye una breve explicación de los módulos implementados para el proyecto, y además, se incluyen imágenes y figuras que corresponden al diseño digital hecho de forma manual. No se hace uso de un orden particular para describir cada módulo, ya que estos fueron programados en un orden diferente al orden con el que se diseñaron originalmente, debido a que varias implementaciones fueron adaptadas de forma diferente en el código, lo cual es producto de la modularidad del lenguaje *Verilog*.

Para la programación de los módulos se sigue la idea de que primero se definen variables de entrada, luego las salidas, luego las señales internas y finalmente la lógica combinacional que utiliza los elementos anteriores.

1.1. `sumador_1bit.v`

Este módulo es uno de los ejemplos básicos de la lógica combinacional aplicada en operaciones binarias, también se le llama “Half Adder” porque no acepta como entrada un C_{in} . Solo requiere dos compuertas y el código en *Verilog* es de los más simples del proyecto.

1.2. `full_adder_1bit.v`

Este módulo es programado a partir del módulo de **`sumador_1bit.v`**, pero el diseño combinacional manual realizado difiere un poco, ya que en el código queremos optimizar la reutilización de la mayor cantidad de módulos posibles. La Figura 6 incluye el desarrollo manual del diseño digital.

1.3. `sumador_4bit.v`

Este módulo ya corresponde a una función completa, que es la de sumar dos números de cuatro bits. Usa el módulo **full_adder_1bit.v**, por lo que también usa el módulo **sumador_1bit.v**, ya que el primero depende del segundo. Las Figuras 7 y 12 incluyen el desarrollo manual del diseño digital.

1.4. `restador_4bit.v`

Este módulo se encarga de restar dos números de 4 bits, utilizando el método de complemento a 2, el cual permite realizar una resta reutilizando la lógica de un sumador. Para esto, se niega cada bit del segundo operando y se suma junto con un acarreo de entrada fijo en 1, logrando así el efecto de una resta sin necesidad de diseñar un circuito completamente nuevo. Es un buen ejemplo de cómo un bloque combinacional más simple, como el sumador, puede ser reutilizado para construir una operación distinta. En los anexos se incluyen las Figuras 8 y 13 tienen el desarrollo manual del diseño digital.

1.5. `restador_inverso_4bit.v`

Este módulo cumple una función similar al `restador_4bit`, pero calcula la resta en el orden inverso, es decir, obtiene el resultado de B menos A en lugar de A menos B. Al igual que el restador convencional, se apoya en el método de complemento a 2, negando el operando correspondiente y sumando un acarreo de entrada fijo en 1 para lograr el efecto de la resta a partir de un sumador. Este módulo resulta necesario dentro de la calculadora para poder ofrecer ambas variantes de resta como operaciones seleccionables por el usuario. En los anexos se incluyen las Figuras 9 y 14 que tienen el desarrollo manual del diseño digital.

1.6. `magnitud_4bit.v`

Este módulo se encarga de extraer el signo y la magnitud de un número de 4 bits representado en complemento a 2. El signo se obtiene directamente del bit más significativo del número, mientras que la magnitud se calcula utilizando un restador para obtener la versión positiva del valor en caso de que este sea negativo, seleccionando luego entre el valor original o su versión convertida según corresponda. Este módulo es fundamental para la etapa de visualización, ya que permite mostrar los números de forma separada en signo y valor absoluto en los displays de la calculadora. En los anexos se incluyen las Figuras 10 que tienen el diagrama del diseño digital.

1.7. `display_hex_7seg.v`

Este módulo se encarga de decodificar un número de 4 bits para mostrarlo en formato hexadecimal (de 0 a F) en un display de siete segmentos. Para lograr esto, se detectan todas las combinaciones posibles del valor de entrada mediante compuertas AND, y luego se determina, a través de compuertas OR, qué segmentos deben encenderse según el dígito que corresponda representar. Este módulo es un ejemplo clásico de decodificador combinacional, ampliamente utilizado en el diseño digital para la visualización de valores numéricos. En los anexos se incluyen las Figuras 23, 24, 25, 26, 27 y 28 que tienen el desarrollo manual del diseño digital.

1.8. `display_signo_7seg.v`

Este módulo se encarga de mostrar el signo de un número en un display de siete segmentos, utilizando únicamente el segmento central para representar el símbolo cuando el número es negativo. Si el número es positivo, todos los segmentos permanecen apagados, dejando el display vacío. Es uno de los módulos más simples del proyecto, ya que solo requiere un buffer para el segmento central y compuertas AND para mantener el resto de los segmentos en 0. En los anexos se incluyen figuras que tienen el desarrollo manual del diseño digital.

1.9. `visualizador_valor_4bit.v`

Este módulo integra los bloques encargados de calcular el signo y la magnitud de un número de 4 bits junto con los decodificadores necesarios para mostrarlo en los displays de siete segmentos. Su función es coordinar la comunicación entre el módulo `magnitud_4bit` y los módulos `display_signo_7seg` y `display_hex_7seg`, entregando como salida las señales listas para controlar ambos displays. De esta forma, este módulo actúa como una capa intermedia que simplifica la visualización de cualquier número de 4 bits dentro del resto del proyecto. En los anexos se incluyen figuras que tienen el desarrollo manual del diseño digital.

1.10. `shift_right_4bit.v`

Este módulo se encarga de desplazar un número de 4 bits hacia la derecha, según la cantidad de posiciones indicada por un valor de 2 bits, rellendo con ceros desde la izquierda. Para lograr esto, cada bit de salida se construye seleccionando, mediante compuertas AND y OR, el bit de entrada correspondiente según la cantidad de desplazamiento solicitada, funcionando de manera similar a un multiplexor. Este módulo permite incorporar operaciones de desplazamiento como parte de las funciones disponibles en la calculadora. En los anexos se incluyen las Figuras 15 y 16 que tienen el desarrollo manual del diseño digital.

1.11. `shift_left_4bit.v`

Este módulo se encarga de desplazar un número de 4 bits hacia la izquierda, según la cantidad de posiciones indicada por un valor de 2 bits, rellendo con ceros desde la derecha. Funciona de manera similar a `shift_right_4bit`. Junto con el módulo anterior, este bloque completa el conjunto de operaciones de desplazamiento disponibles dentro de la calculadora. En los anexos se incluyen las Figuras 17 y 18 que tienen el desarrollo manual del diseño digital.

1.12. `leds_operacion.v`

Este módulo se encarga de mostrar en los leds de la placa el código de la operación actualmente seleccionada por el usuario. Su funcionamiento es directo, ya que cada bit del código de operación se conecta a un led mediante un buffer, sin necesidad de ningún tipo de procesamiento adicional. Es uno de los módulos más simples del proyecto, y permite que el usuario tenga una referencia visual inmediata de qué operación está seleccionada mientras interactúa con la calculadora. En los anexos se incluyen figuras que tienen el desarrollo manual del diseño digital.

1.13. `visualizacion_estado.v`

Este módulo se encarga de determinar qué valor debe mostrarse en los displays de siete segmentos en cada momento, dependiendo del estado actual de la calculadora. Según corresponda, selecciona entre el operando A, el operando B o el resultado de la operación, utilizando compuertas AND y OR a modo de multiplexor, y apaga completamente los displays en caso de que el estado no requiera mostrar ningún valor. Para obtener las señales finales de los displays, este módulo se apoya en el visualizador_valor_4bit, encargado de calcular el signo y la magnitud del valor seleccionado. En los anexos se incluyen figuras que tienen el desarrollo manual del diseño digital.

1.14. `selector_operacion_4bit.v`

Este módulo se encarga de elegir, entre los resultados de todas las operaciones calculadas en paralelo (suma, resta, resta invertida, shift left y shift right), aquel que corresponde según el código de operación indicado por el usuario. Para lograr esto, se decodifica el código de 3 bits recibido y se utiliza dicha decodificación para seleccionar, mediante compuertas AND y OR a modo de multiplexor, el resultado correcto entre las distintas opciones disponibles. Este módulo es clave dentro de la calculadora, ya que centraliza la decisión de qué operación finalmente se entrega como resultado. En los anexos se incluyen figuras que tienen el desarrollo manual del diseño digital.

1.15. selector_op2_4bit.v

Este módulo se encarga de determinar cuál será el segundo operando utilizado en el cálculo de la operación, pudiendo ser el valor ingresado directamente por el usuario (OP2) o el resultado de la operación anterior, en caso de que se desee encadenar cálculos. Para esto, utiliza la señal `usar_anterior` para seleccionar, mediante compuertas AND y OR a modo de multiplexor, cuál de los dos valores corresponde entregar como salida. Este módulo permite que la calculadora ofrezca la funcionalidad de reutilizar resultados previos como parte de nuevas operaciones. En los anexos se incluyen figuras que tienen el desarrollo manual del diseño digital.

1.16. registro_operacion_3bit.v

Este módulo corresponde a un registro secuencial de 3 bits encargado de almacenar el código de la operación seleccionada por el usuario. A diferencia de un contador convencional, este registro recorre un ciclo de seis estados (de 0 a 5), permitiendo incrementar o disminuir el valor almacenado mediante señales de control, además de mantenerse en su valor actual o resetearse a 0 según corresponda. Este módulo introduce el uso de lógica secuencial dentro del proyecto, sincronizando sus cambios de estado con la señal de reloj. En los anexos se incluyen figuras que tienen el desarrollo manual del diseño digital.

1.17. registro_resultado_4bit.v

Este módulo corresponde a un registro secuencial de 4 bits encargado de almacenar el resultado de la calculadora, manteniéndolo estable hasta que se realice un nuevo cálculo. Su valor solo se actualiza cuando la señal `guardar` está activa, permitiendo cargar el nuevo resultado. En caso contrario, el registro conserva su valor actual, o se resetea a 0 si la señal de `reset` está activa. Este módulo permite que el resultado permanezca disponible incluso después de haber sido calculado, tanto para su visualización como para su posible reutilización en operaciones futuras. En los anexos se incluyen figuras que tienen el desarrollo manual del diseño digital.

1.18. `registro_ajustable_4bit.v`

Este módulo corresponde a un registro secuencial de 4 bits encargado de almacenar el valor de uno de los operandos ingresados por el usuario. Utilizando internamente un sumador_4bit y un restador_4bit, este registro permite incrementar o disminuir en 1 el valor almacenado mediante señales de control, además de mantenerse en su valor actual o resetearse a 0 según corresponda. Este módulo reutiliza bloques combinacionales ya diseñados previamente para construir un elemento secuencial más complejo, siendo utilizado para ingresar ambos operandos de la calculadora. En los anexos se incluyen las figuras que tienen el desarrollo manual del diseño digital.

1.19. `botones_goboard.v`

Este módulo se encarga de procesar las señales físicas provenientes de los cuatro botones de la placa, transformándolas en señales limpias y utilizables por el resto del sistema. Para esto, implementa sincronizadores de doble registro que evitan problemas de metaestabilidad, un contador que genera una señal periódica utilizada para muestrear los botones a una frecuencia menor, y detección de flancos de subida para generar pulsos de un solo ciclo en el caso de los botones de incrementar, disminuir y confirmar. Este módulo resulta esencial para lograr una interacción confiable entre el usuario y la calculadora, filtrando los rebotes propios de los botones físicos mediante lógica secuencial. En los anexos se incluye la Figura 11 que tiene el diseño final implementado en código.

1.20. control_valores.v

Este módulo se encarga de determinar cuál de los tres valores de entrada (la operación, el operando A o el operando B) debe estar habilitado para ser modificado en un momento dado, según el estado actual de la calculadora. Para esto, decodifica el estado de 2 bits recibido y activa, mediante compuertas AND, únicamente la señal correspondiente al valor que debe poder editarse en ese momento. Este módulo es clave para coordinar que los botones de incrementar y disminuir afecten siempre al valor correcto, evitando que se modifiquen accidentalmente otros registros del sistema.

1.21. control_estados.v

Este módulo corresponde a un registro secuencial de 2 bits encargado de controlar en qué etapa se encuentra la calculadora en un momento dado: ingreso de la operación, ingreso del operando A, ingreso del operando B, o visualización del resultado. Su valor avanza al siguiente estado cada vez que se presiona el botón de confirmar, siguiendo un ciclo cerrado que vuelve al estado inicial una vez alcanzada la etapa de resultado, o se resetea a 0 en caso de que la señal de reset esté activa. Este módulo actúa como el controlador principal del flujo de interacción de la calculadora, definiendo el comportamiento general del sistema en cada momento.

1.22. control_entrada.v

Este módulo integra los bloques encargados de gestionar el ingreso de datos por parte del usuario, coordinando el control del estado actual de la calculadora junto con los registros que almacenan la operación seleccionada y ambos operandos. A partir del estado entregado por control_estados, utiliza control_valores para determinar cuál registro debe estar habilitado en cada momento, y conecta dicha información con los módulos registro_operacion_3bit, registro_ajustable_4bit (instanciado dos veces, una para cada operando) para permitir su modificación mediante los botones de incrementar y disminuir. Este módulo actúa como una capa intermedia que organiza todo el proceso de ingreso de datos dentro de la calculadora.

1.23. calculadora_sistema.v

Este módulo integra el proceso de ingreso de datos con el proceso de cálculo de la operación, coordinando el momento exacto en que corresponde ejecutar dicho cálculo. Utilizando el módulo control_entrada para gestionar la operación y ambos operandos, este módulo genera la señal de guardar en el instante en que el usuario confirma el ingreso del segundo operando, activando así el módulo calculadora_core_4bit para que realice el cálculo correspondiente y almacene el resultado. Este módulo representa un nivel superior de integración dentro del proyecto, conectando la etapa de entrada del usuario con la etapa de procesamiento de la calculadora.

1.24. calculadora_core_4bit.v

Este módulo constituye el núcleo de procesamiento de la calculadora, encargado de calcular en paralelo todas las operaciones disponibles (suma, resta, resta invertida, shift left y shift right) a partir de los dos operandos recibidos, para luego seleccionar cuál de estos resultados corresponde entregar según el código de operación indicado. Además, mediante el módulo selector_op2_4bit, permite decidir si el segundo operando utilizado en el cálculo corresponde al valor ingresado por el usuario o al resultado de una operación anterior, para posteriormente almacenar el resultado final utilizando registro_resultado_4bit. Este módulo reúne gran parte de los bloques combinatoriales diseñados previamente en el proyecto, integrándolos en una única unidad de cálculo.

1.25. top.v

Este módulo corresponde al nivel superior del proyecto, encargado de conectar todos los bloques diseñados con los pines físicos de la FPGA Lattice iCE40 HX1K, integrada en la Nandland Go Board. En primer lugar, genera una señal de reset automático durante el primer ciclo de reloj tras el encendido, para asegurar un estado inicial correcto en todo el sistema. Luego, mediante `botones_goboard`, traduce las señales físicas de los botones en señales de control utilizables, las cuales son entregadas a `calculadora_sistema`, módulo que concentra toda la lógica de la calculadora. Finalmente, conecta las salidas obtenidas con los leds y displays de siete segmentos físicos de la placa, negando las señales correspondientes a los displays, ya que estos son de tipo *active-low*. Este módulo representa la integración final de todos los bloques del proyecto en un único sistema funcional, listo para ser programado sobre la FPGA.

2. Funcionamiento de la FPGA y Compilamiento de Código

Una FPGA (*Field Programmable Gate Array*) es un circuito integrado que, a diferencia de un procesador convencional, no ejecuta instrucciones de manera secuencial, sino que está compuesto por una gran cantidad de bloques lógicos configurables que pueden interconectarse entre sí para formar cualquier circuito digital deseado. Esto significa que, en lugar de programar una FPGA en el sentido tradicional de escribir instrucciones para un procesador, lo que se hace es describir la estructura del circuito que se desea implementar, y esa descripción se traduce físicamente en las conexiones internas del chip.

En particular, este proyecto utiliza la *FPGA Lattice iCE40 HX1K*, integrada en la placa de desarrollo *Nandland Go Board*. Esta FPGA está compuesta internamente por múltiples bloques lógicos programables, cada uno de los cuales puede configurarse para implementar pequeñas funciones lógicas combinacionales o secuenciales. Al interconectar estos bloques entre sí, siguiendo la estructura descrita en el código *Verilog* del proyecto, es posible construir circuitos mucho más complejos, como los distintos módulos que conforman la calculadora. Además, la *Go Board* incluye periféricos integrados, como botones, leds y displays de siete segmentos, los cuales son utilizados directamente por este proyecto para permitir la interacción con el usuario.

Para poder llevar el código *Verilog* desde su forma escrita hasta su implementación física en la FPGA, es necesario pasar por un flujo de herramientas de código abierto, agrupadas dentro del paquete conocido como *oss-cad-suite*. Este flujo se compone de varias etapas, cada una realizada por una herramienta específica.

En primer lugar, se utiliza *Yosys*, el cual actúa como sintetizador. Su función es leer el código *Verilog* del proyecto y transformarlo en una representación a nivel de compuertas lógicas, conocida como netlist. Esta netlist describe el circuito en términos de compuertas básicas y sus conexiones, sin hacer aún referencia a la ubicación física dentro del chip.

A continuación, se utiliza *nextpnr-ice40*, encargado del proceso de *place and route*. Esta herramienta toma la netlist generada por Yosys y determina en qué posición exacta de la FPGA se ubicará cada uno de los bloques lógicos necesarios, además de definir cómo se conectarán entre sí utilizando los recursos de enrutamiento disponibles dentro del chip, respetando las restricciones físicas y de temporización del dispositivo.

Una vez que el circuito ya ha sido ubicado y enrutado dentro de la FPGA, se utiliza *icepack*, cuya función es convertir el resultado del proceso anterior en un archivo binario, conocido como bitstream. Este archivo contiene toda la información necesaria para configurar internamente los bloques lógicos y las conexiones de la FPGA, en el formato específico que el chip es capaz de interpretar.

Finalmente, se utiliza *iceprog* para cargar el bitstream generado directamente en la FPGA, a través de la conexión USB de la Go Board. Este paso programa físicamente el chip, configurando sus bloques internos según lo definido por el circuito diseñado, permitiendo así que la calculadora quede operativa y lista para ser utilizada. De esta manera, el flujo completo permite transformar una descripción escrita en *Verilog* en un circuito digital funcional, implementado físicamente sobre la *FPGA Lattice iCE40 HX1K*.

3. Testbenching de la FPGA

Para corroborar el correcto funcionamiento de los módulos diseñados para la FPGA, se realiza un proceso de verificación conocido como *testbenching*. Un testbench es un módulo de Verilog (o SystemVerilog) que no forma parte del hardware final, sino que se utiliza exclusivamente en simulación, con el objetivo de aplicar distintos valores de entrada al circuito diseñado y comprobar automáticamente si las salidas obtenidas coinciden con los resultados esperados.

En este proyecto, la verificación se realiza utilizando un archivo entregado por uno de los ayudantes del ramo, llamado `calculadora_4bits_tb_basico.sv`. Este testbench se compila junto con los módulos del proyecto utilizando el programa *oss-cad-suite*, lo que permite ejecutar la simulación y generar los resultados correspondientes.

El testbench funciona instanciando el módulo principal de la calculadora como *Device Under Test* (DUT), y generando internamente una señal de reloj que alterna su valor cada 5 unidades de tiempo, simulando así el comportamiento real del sistema. Además, se define una tarea llamada `pulso_ejecutar`, encargada de activar la señal `ejecutar` durante un ciclo de reloj, imitando la acción de confirmar una operación tal como ocurriría al presionar un botón físico en la FPGA.

Para realizar las pruebas propiamente tales, se utiliza una tarea llamada `probar`, la cual recibe como parámetros el código de la operación a testear, los dos operandos, el resultado esperado y un nombre descriptivo de la prueba. Esta tarea configura las entradas del sistema, ejecuta la operación mediante `pulso_ejecutar`, y compara el resultado obtenido con el resultado esperado. En caso de que ambos valores coincidan, se imprime un mensaje indicando que la prueba fue exitosa (**PASS**); en caso contrario, se imprime un mensaje de error (**FAIL**) junto con los detalles de la prueba, y la simulación se detiene inmediatamente mediante la instrucción `$fatal`.

Dentro del bloque principal del testbench, se inicializan todas las señales en su estado por defecto y se generan los archivos necesarios para la visualización de las señales mediante las instrucciones `$dumpfile` y `$dumpvars`, los cuales permiten registrar la evolución de todas las señales internas del sistema a lo largo de la simulación. Posteriormente, se ejecutan diversas pruebas sobre las operaciones de suma y resta, incluyendo casos con números negativos representados en complemento a 2, así como casos que generan overflow, verificando que en dichas situaciones solo se conserven los cuatro bits menos significativos del resultado, tal como se especifica en los requerimientos del proyecto.

Una vez finalizada la simulación, el archivo generado puede visualizarse utilizando el programa **GTKWave**, el cual permite observar gráficamente la evolución de cada una de las señales del circuito a lo largo del tiempo, facilitando la identificación de errores de diseño en caso de que alguna prueba falle. De manera complementaria, los mensajes incorporados dentro del propio testbench permiten verificar el resultado de cada prueba de forma directa en la consola, sin necesidad de inspeccionar visualmente las señales, en caso de que solo se busque confirmar que todas las pruebas fueron superadas correctamente.

Finalmente, se presentan las Figuras 2 y 3 que muestran los resultados obtenidos durante la ejecución del testbench.

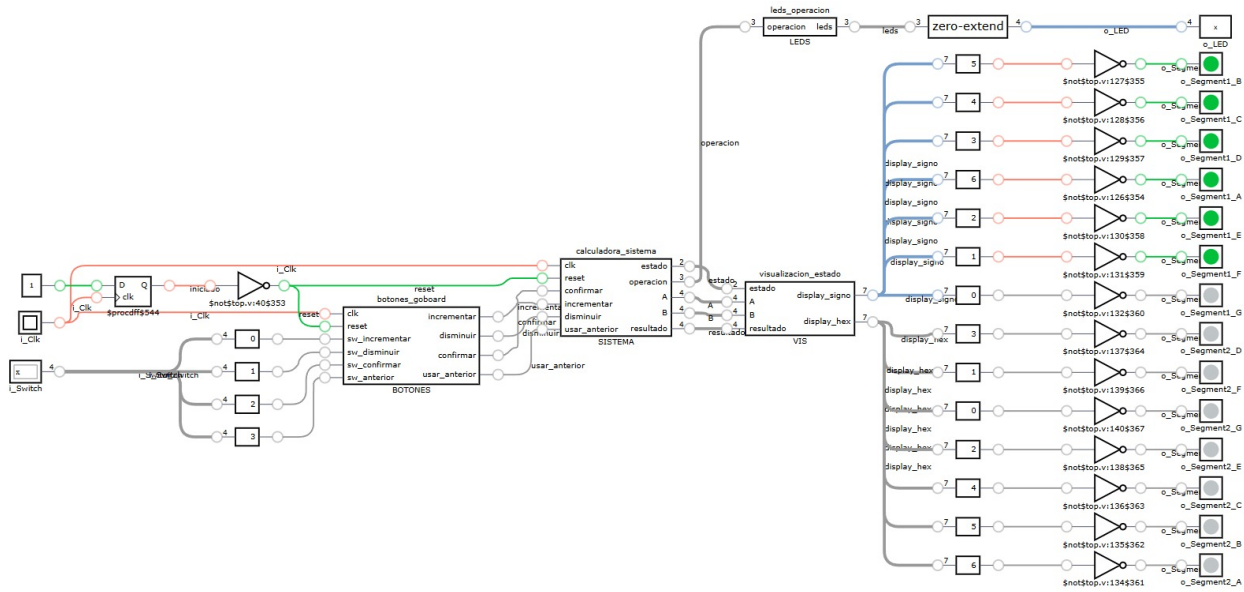


Figura 1: Arquitectura Completa de la FPGA

Fuente: Elaboración Propia

```
[OSS CAD Suite] C:\Users\lancenterstore\Desktop\Proyecto1_Arqui>iverilog -g2012 -s calculadora_4bits_tb_basico -o test_oficial -c sources.f

[OSS CAD Suite] C:\Users\lancenterstore\Desktop\Proyecto1_Arqui>vvp test_oficial
VCD info: dumpfile calculadora_4bits_tb_basico.vcd opened for output.
PASS suma 3 + 4 = 7: resultado = 0111
PASS suma -2 + 3 = 1: resultado = 0001
PASS suma con overflow 7 + 3: resultado = 1010
PASS resta 5 - 2 = 3: resultado = 0011
PASS resta 2 - 5 = -3: resultado = 1101
PASS resta -5 - (-2) = -3: resultado = 1101
LOS TESTS PASARON!
tb\calculadora_4bits_tb_basico.sv:89: $finish called at 136000 (1ps)

[OSS CAD Suite] C:\Users\lancenterstore\Desktop\Proyecto1_Arqui>
```

Figura 2: Resultado del testbench por consola

Fuente: Elaboración Propia

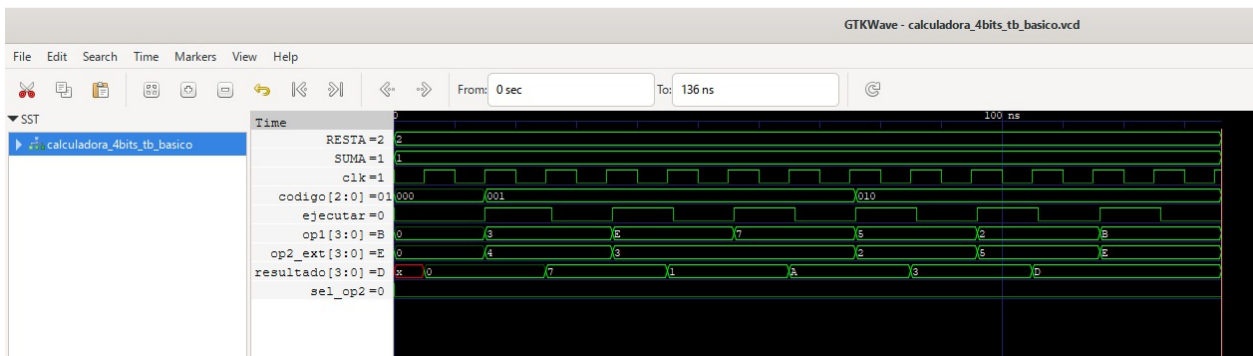


Figura 3: Resultado del testbench por GTKwave

Fuente: Elaboración Propia

d: se $\tilde{0}$ complemento A_2

A_3	A_2	A_1	A_0	B_3	B_2	B_1	B_0
0	0	0	0	0	0	0	0
0	0	0	1	1	1	1	1
0	0	1	0	1	1	1	0
0	0	1	1	1	1	0	1
0	1	0	0	1	1	0	0
0	1	0	1	1	0	1	1
0	1	1	0	1	0	1	0
0	1	1	1	1	0	0	1
1	0	0	0	1	0	0	0
1	0	0	1	0	1	1	1
1	0	1	0	0	1	1	0
1	0	1	1	0	1	0	1
1	1	0	0	0	1	0	0
1	1	0	1	0	0	1	1
1	1	1	0	0	0	1	0
1	1	1	1	0	0	0	1

$A_3 A_2$	00	01	11	10
00				
01				
11				
10				

$B_3 B_2$	00	01	11	10
00	0	1	1	1
01	1	1	1	1
11	0	0	0	0
10	1	0	0	0

q_1 0 1 0 1 q_2 0 0 0 1 q_3 0 0 1 1 q_4 1 0 0 0
 0 1 0 1 0 0 1 1 0 0 1 0
 0 1 1 1 0 1 0 1 0 1 1 1
 0 1 1 0 0 1 1 1 0 1 1 0

$\bar{A}_3 \cdot \bar{A}_2$ $\bar{A}_3 \cdot A_0$ $\bar{A}_3 \cdot A_1$ $A_3 \cdot \bar{A}_1 \cdot \bar{A}_2 \cdot \bar{A}_0$

$B_3 = \bar{A}_2 (A_2 + A_1 + A_0) + A_3 \cdot \bar{A}_2 \cdot \bar{A}_1 \cdot \bar{A}_0$

Figura 4: Desarrollo del Complemento a 2, Parte 1

Fuente: Elaboración Propia

B₂ A₁A₀

B ₂ B ₁	0 0	0 1	1 1	1 0
0 0	0	1	1	1
0 1	1	0	0	0
1 1	1	0	0	0
1 0	0	1	1	1

q₁ 0 0 0 1 q₂ 0 0 1 1 q₃ 0 1 0 0
 0 0 1 1 0 0 1 0 1 1 0 0
 1 0 0 1 1 0 1 1
 1 0 1 1 1 0 1 0

$$B_2 = \bar{A}_2 A_0 + \bar{A}_2 \cdot A_1 + A_2 \bar{A}_1 \bar{A}_0$$

$$B_2 = \bar{A}_2 (A_0 + A_1) + A_2 \cdot \bar{A}_1 \cdot \bar{A}_0$$

B₁ A₁A₀

B ₁ B ₀	0 0	0 1	1 1	1 0
0 0	0	1	0	1
0 1	0	1	0	1
1 1	0	1	0	1
1 0	0	1	0	1

q₁ 0 0 0 1 q₂ 0 0 1 1
 0 1 0 1 0 1 1 0
 1 1 0 1 1 1 1 0
 1 0 0 1 1 0 1 0

$$B_1 = \bar{A}_1 A_0 + A_1 \bar{A}_0$$

$$B_1 = A_1 \oplus A_0$$

A₁A₀

A ₁ A ₀	0 0	0 1	1 1	1 0
0 0	0	1	1	0
0 1	0	1	1	0
1 1	0	1	1	0
1 0	0	1	1	0

q₁ 0 0 0 1
 0 1 0 1
 1 1 0 1
 1 0 0 1
 0 0 1 1
 0 1 1 1
 1 1 1 1
 1 0 1 1

$B_0 = A_0$

Resumen Funciones lógicas complementadas

$$B_3 = \bar{A}_3 (A_2 + A_1 + A_0) + A_3 \cdot \bar{A}_2 \cdot \bar{A}_1 \cdot \bar{A}_0 \quad B_1 = A_1 \oplus A_0$$

$$B_2 = \bar{A}_2 (A_0 + A_1) + A_2 \cdot \bar{A}_1 \cdot \bar{A}_0 \quad B_0 = A_0$$

Figura 5: Desarrollo del Complemento a 2, Parte 2

Fuente: Elaboración Propia

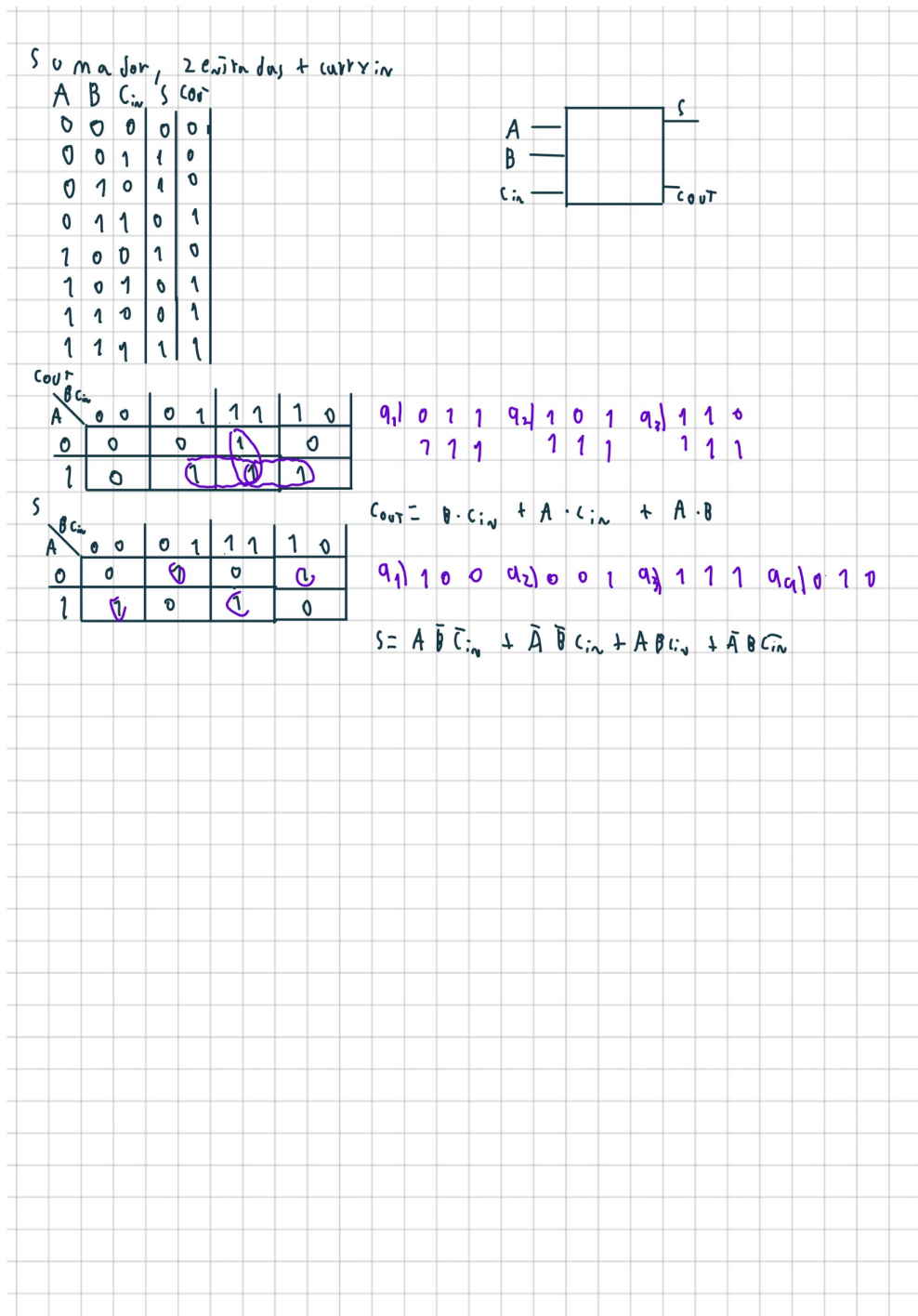


Figura 6: Desarrollo del sumador de 1 bit

Fuente: Elaboración Propia

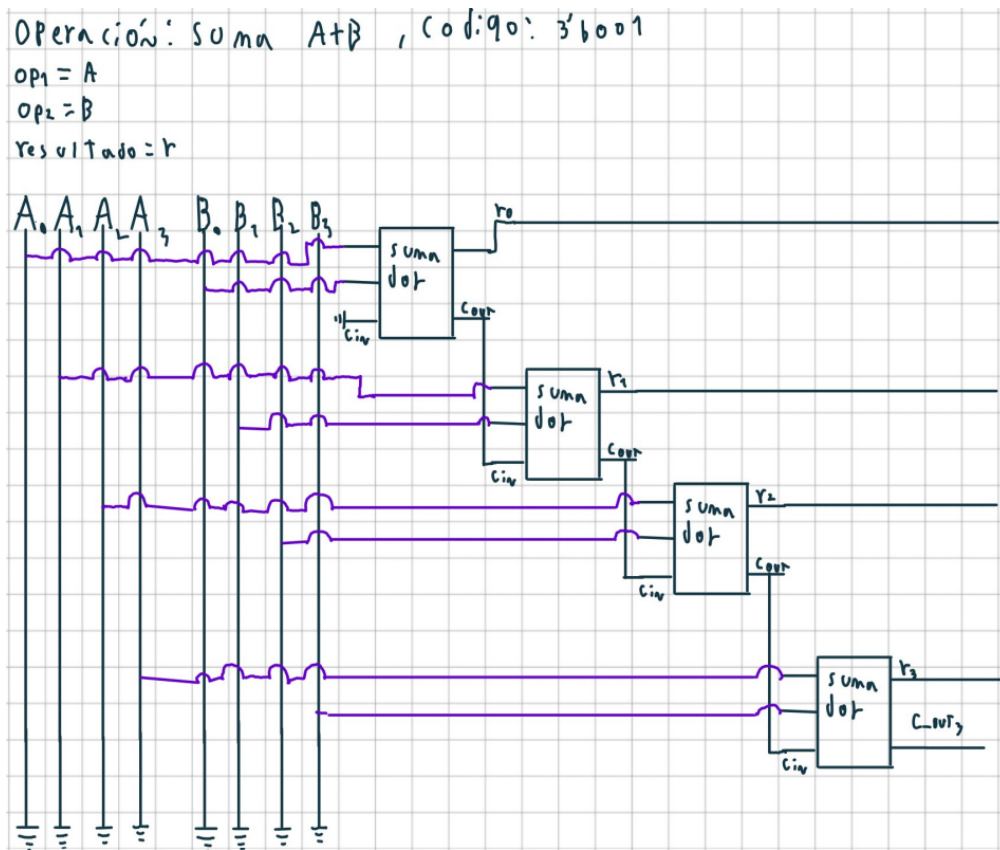


Figura 7: Desarrollo del sumador de 4 bits

Fuente: Elaboración Propia

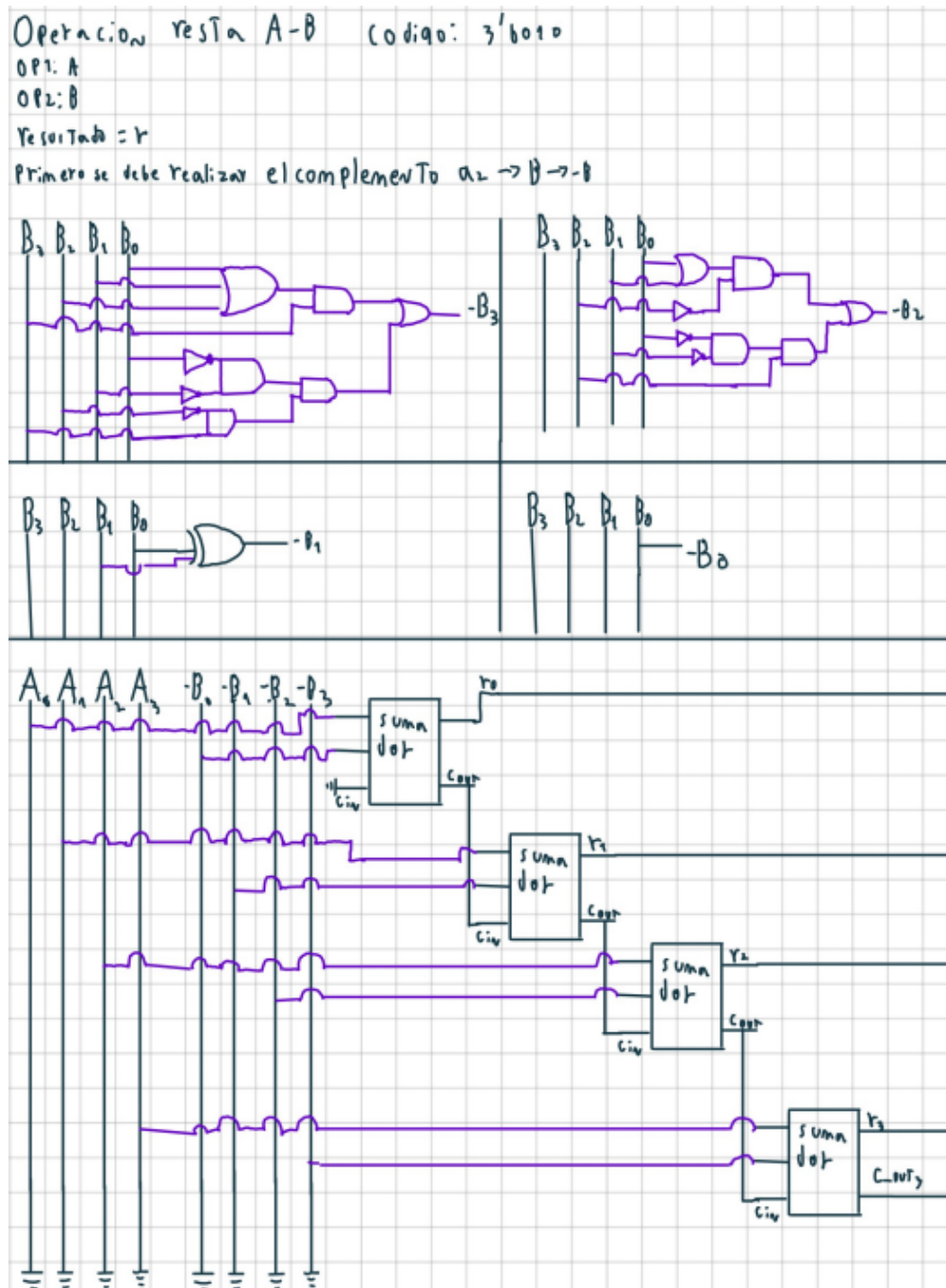


Figura 8: Desarrollo de la resta de 4 bits, $A - B$

Fuente: Elaboración Propia

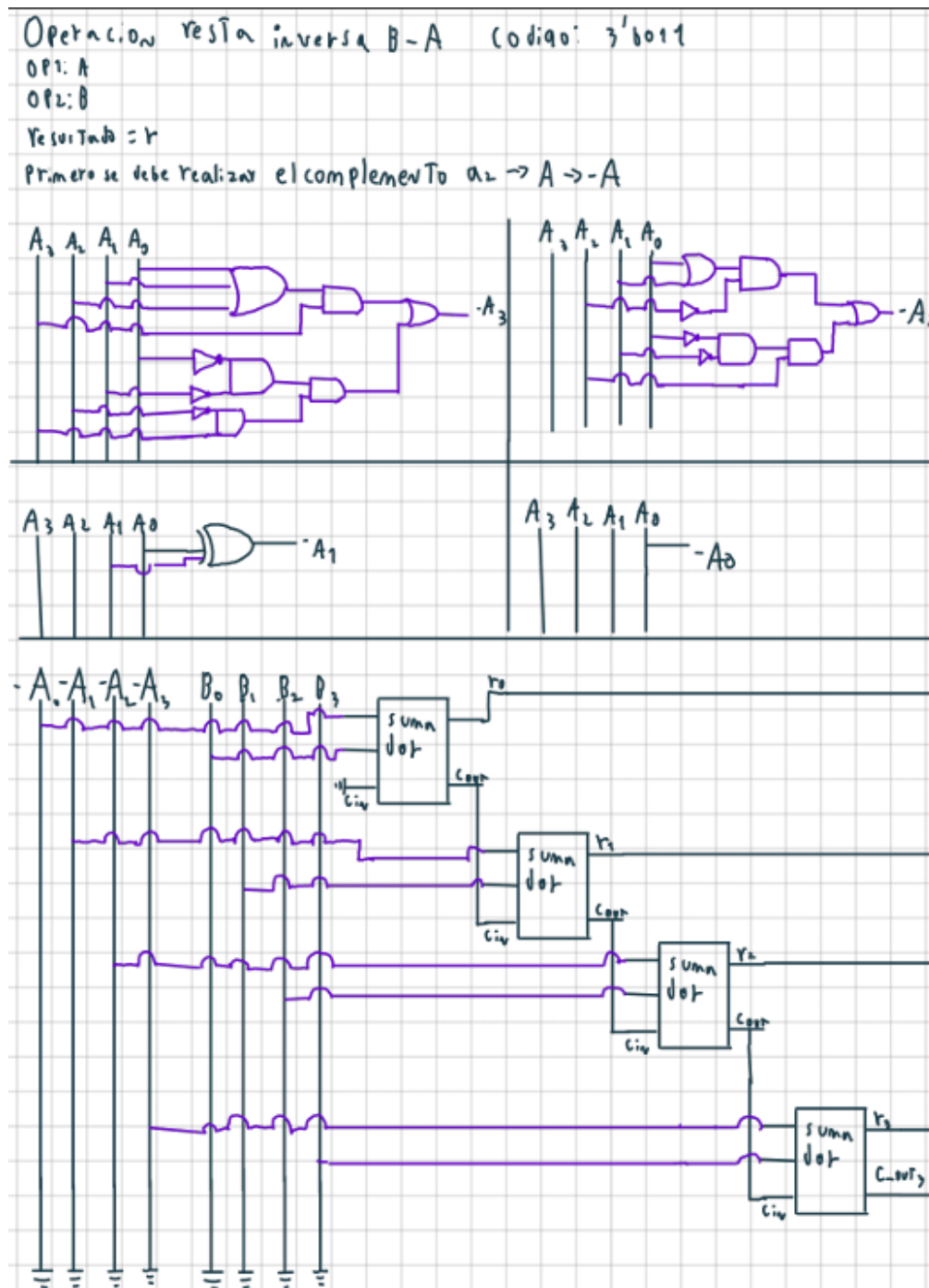


Figura 9: Desarrollo de la resta de 4 bits, $B - A$

Fuente: Elaboración Propia

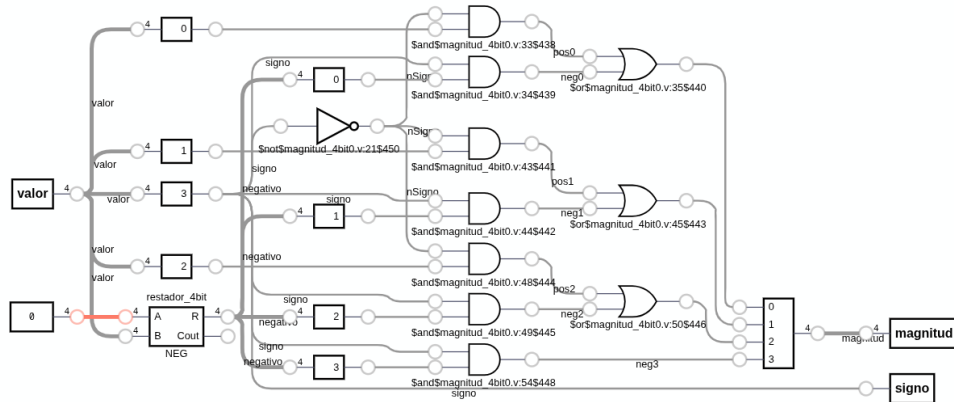


Figura 10: Diagrama del módulo de Magnitud

Fuente: Elaboración Propia

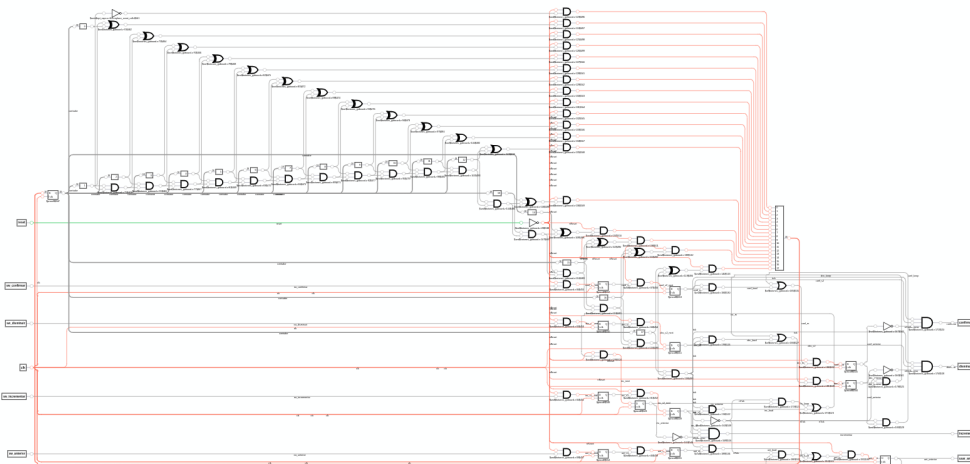


Figura 11: Diagrama del módulo de *botones_goboard*

Fuente: Elaboración Propia

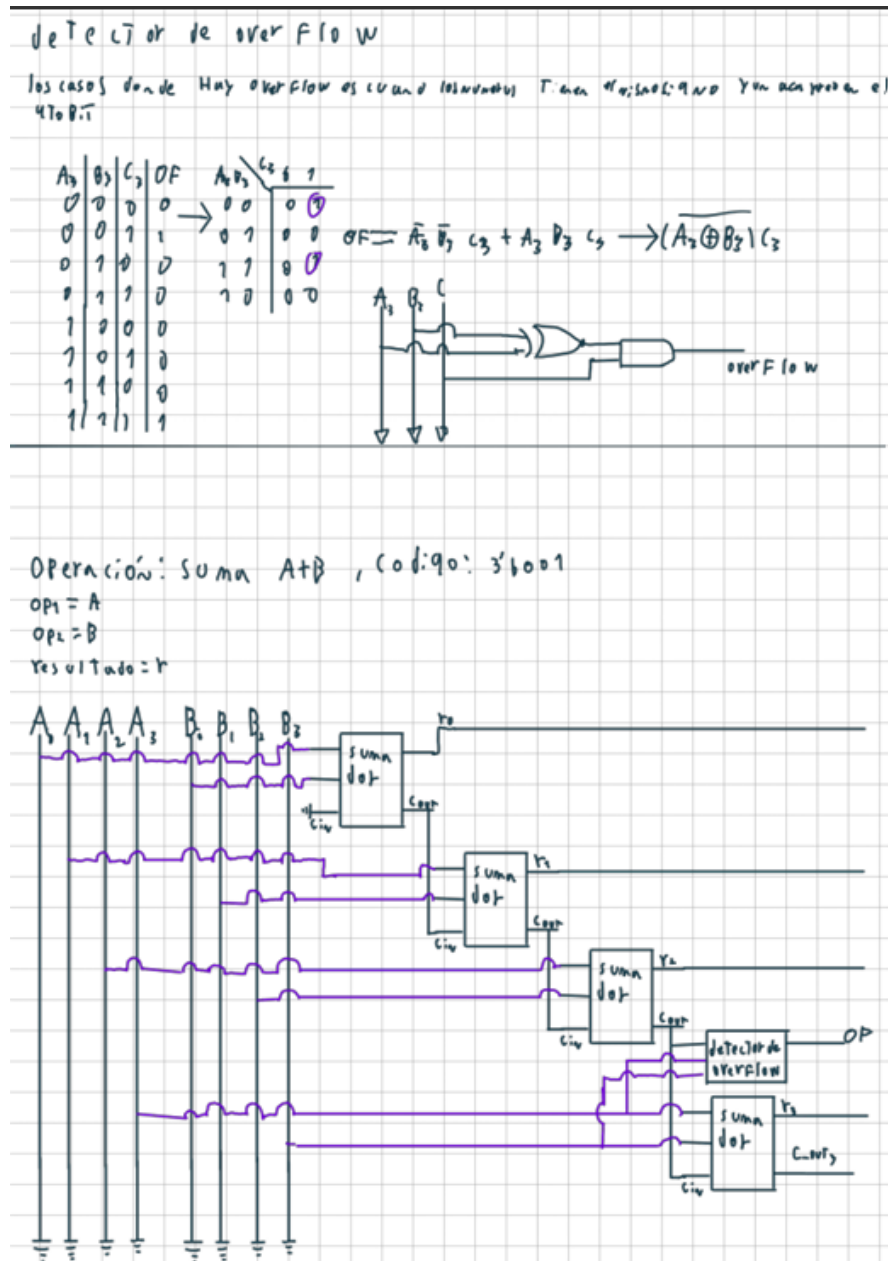


Figura 12: Desarrollo del sumador de 4 bits, considerando overflow

Fuente: Elaboración Propia

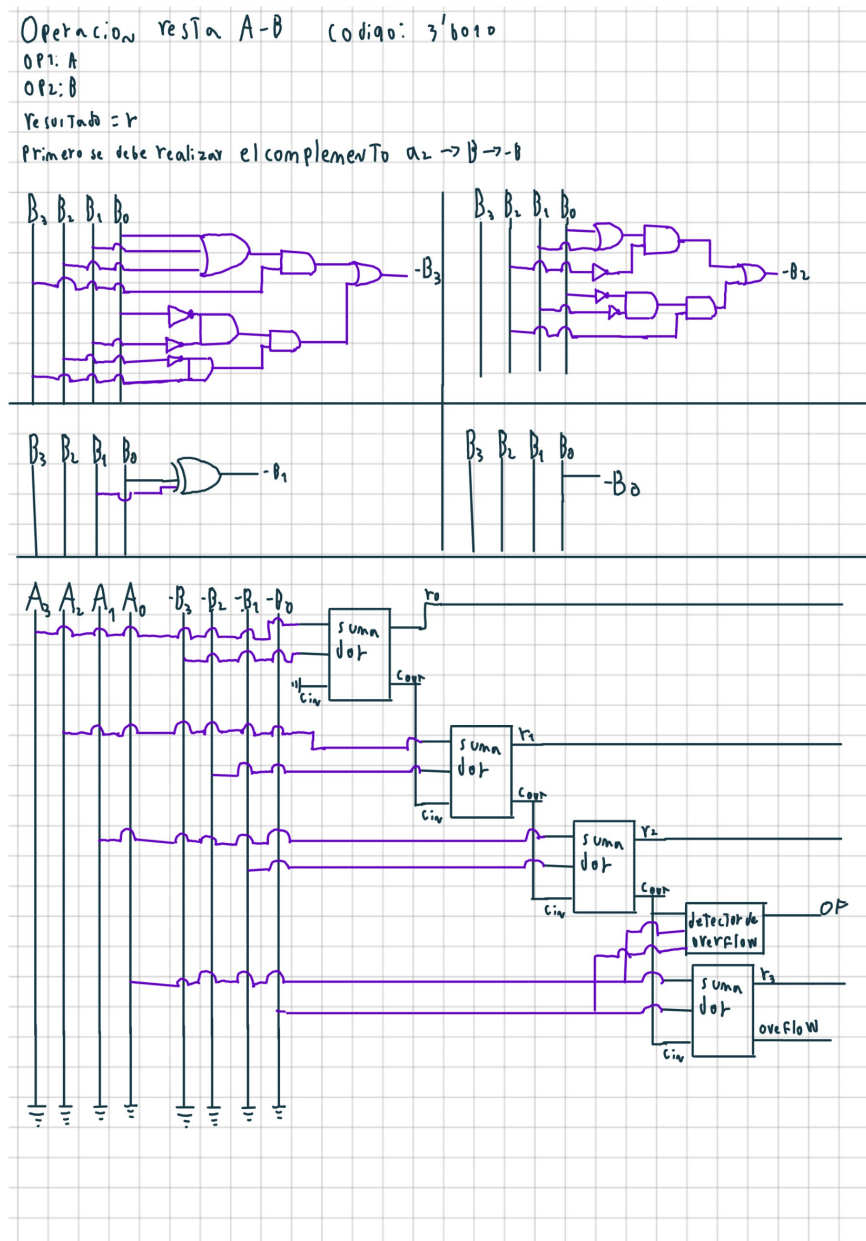


Figura 13: Desarrollo de la resta de 4 bits, $A - B$, considerando overflow

Fuente: Elaboración Propia

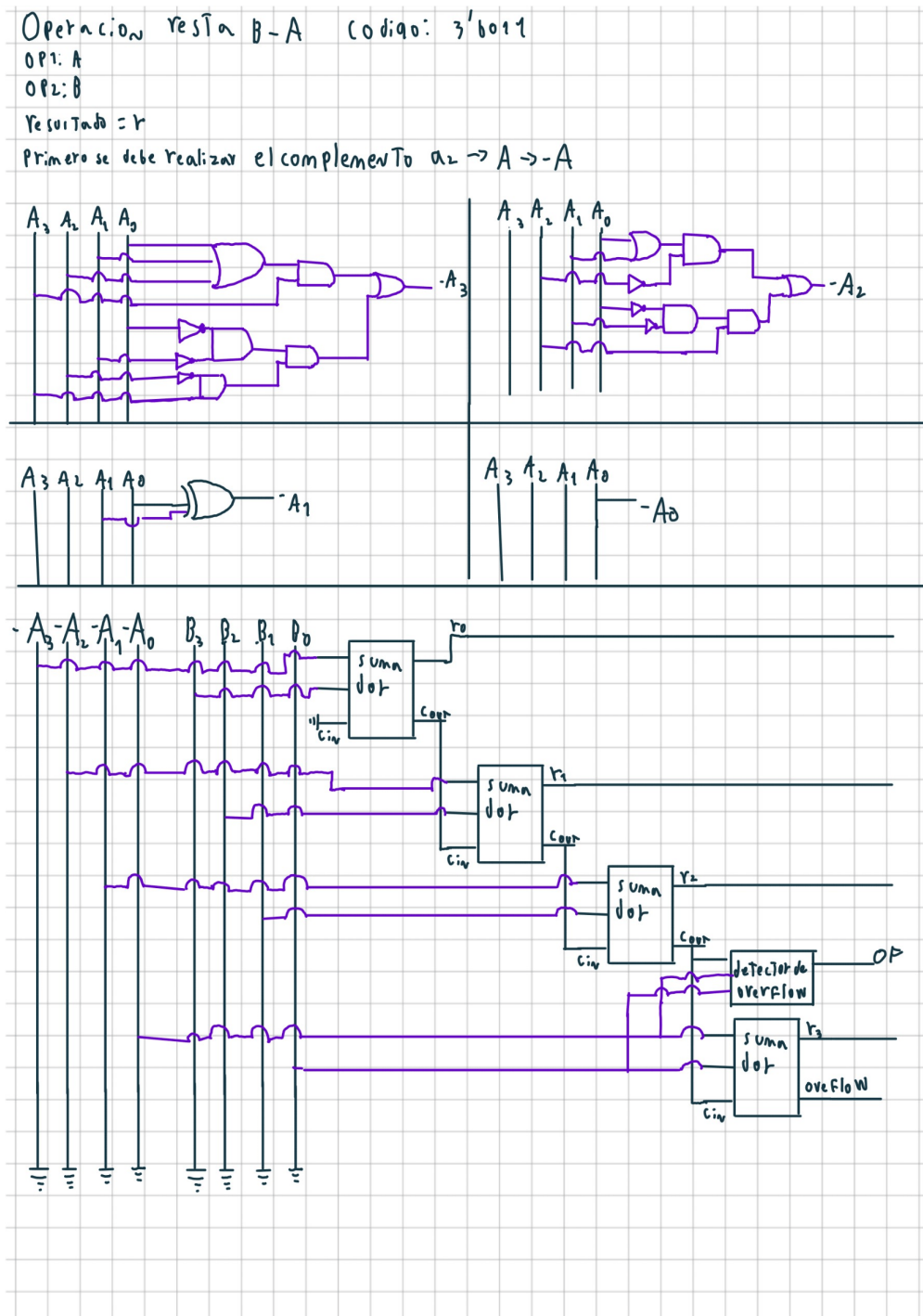


Figura 14: Desarrollo de la resta de 4 bits, $B - A$, considerando overflow

Fuente: Elaboración Propia

B_1	B_0	A_3	A_2	A_1	A_0	r_3	r_2	r_1	r_0
0	0	A_3	A_2	A_1	A_0	A_3	A_2	A_1	A_0
0	1	A_3	A_2	A_1	A_0	0	A_3	A_2	A_1
1	1	A_3	A_2	A_1	A_0	0	0	A_3	A_2
1	0	A_3	A_2	A_1	A_0	0	0	0	A_3

r_0	B_1	B_0	0	1	a_1	$\bar{B}_1 \bar{B}_0 A_0$	a_2	$\bar{B}_1 \bar{B}_0 A_3$
0	0	0	A_0	A_1	$\bar{B}_1 \bar{B}_0 A_1$		$\bar{B}_1 \bar{B}_0 A_2$	
1	0	1	A_2	A_3			$\bar{B}_1 \bar{B}_0 A_3$	

$$r_0 = \bar{B}_1 (\bar{B}_0 A_0 + \bar{B}_0 A_1) + \bar{B}_1 (\bar{B}_0 A_2 + \bar{B}_0 A_3)$$

r_1	B_1	B_0	0	1	a_1	$\bar{B}_1 \bar{B}_0 A_1$	a_2	$\bar{B}_1 \bar{B}_0 A_3$
0	0	0	A_1	A_2	$\bar{B}_1 \bar{B}_0 A_2$			
1	0	1	A_3					

$$r_1 = \bar{B}_1 (\bar{B}_0 A_1 + \bar{B}_0 A_2) + \bar{B}_1 \bar{B}_0 A_3$$

r_2	B_1	B_0	0	1	a_1	$\bar{B}_1 \bar{B}_0 A_2$	r_2	$\bar{B}_1 (\bar{B}_0 A_2 + \bar{B}_0 A_3)$
0	0	0	A_2	A_3	$\bar{B}_1 \bar{B}_0 A_3$			
1	0	1						

r_3	B_1	B_0	0	1	a_1	$\bar{B}_1 \bar{B}_0 A_3$
0	0	0	A_3	0		
1	0	1	0	0		

$$r_3 = \bar{B}_1 \bar{B}_0 A_3$$

Figura 15: Desarrollo de la función Shiftright, parte 1

Fuente: Elaboración Propia

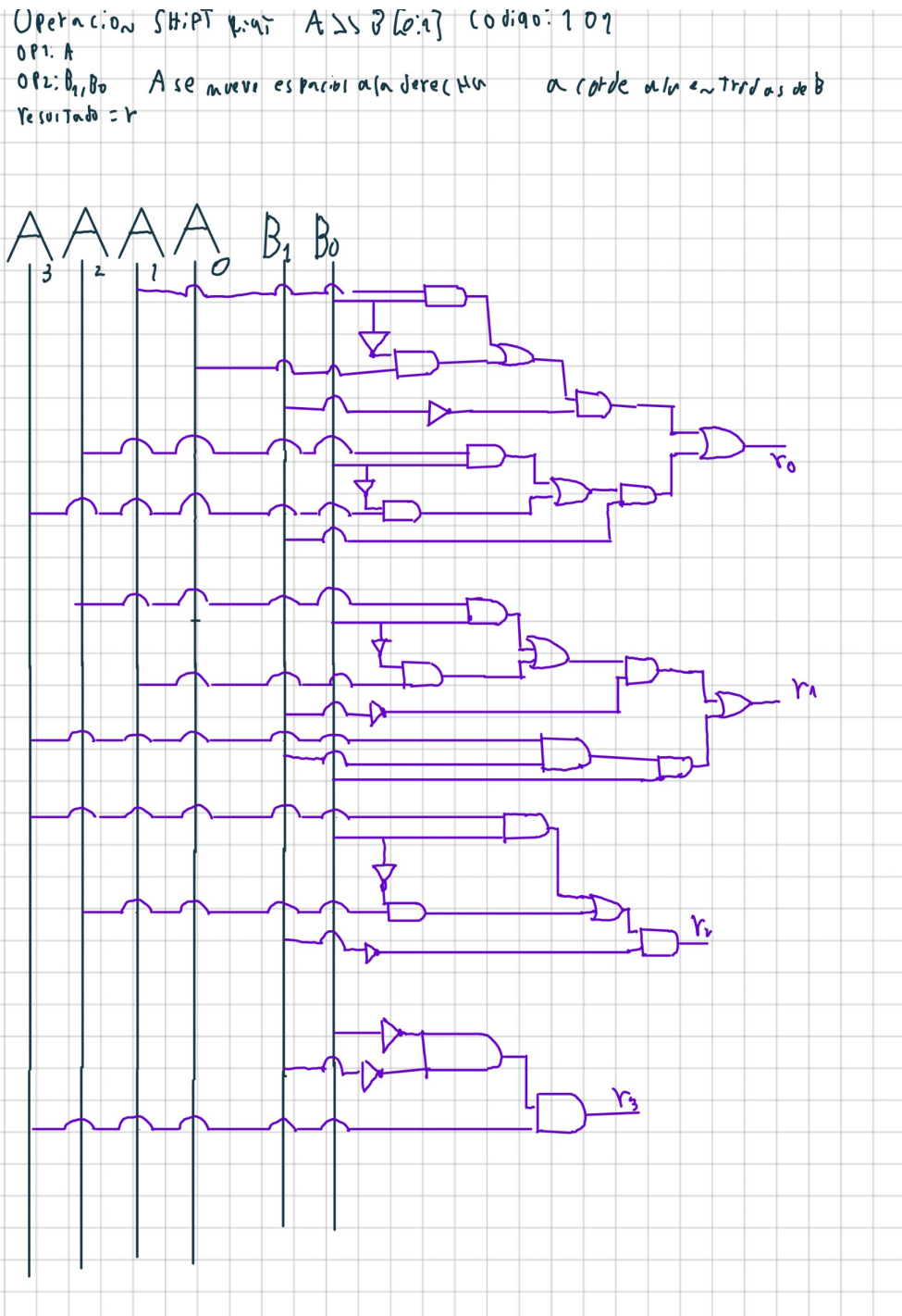


Figura 16: Desarrollo de la función Shiftright, parte 2

Fuente: Elaboración Propia

Queremos que nuestras entradas A_i , se muevan de 1 a 3 puestos a la derecha, en base a los valores de nuestras entradas B_0, B_1

nos que darían entonces

B_1	B_0	A_3	A_2	A_1	A_0	r_3	r_2	r_1	r_0
0	0	A_3	A_2	A_1	A_0	A_3	A_2	A_1	A_0
0	1	A_3	A_2	A_1	A_0	A_2	A_1	A_0	0
1	1	A_3	A_2	A_1	A_0	A_1	A_0	0	0
1	0	A_3	A_2	A_1	A_0	A_0	0	0	0

Realizamos los mapas de Karnaugh

$$r_0 \begin{array}{c|cc} B_1 & B_0 & \\ \hline 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 1 & 0 \\ 1 & 0 & 0 \end{array} \quad a_1: B_1 B_0 \cdot A_0 = r_0$$

$$r_1 \begin{array}{c|cc} B_1 & B_0 & \\ \hline 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 1 & 0 \\ 1 & 0 & 0 \end{array} \quad a_1: \bar{B}_1 \bar{B}_0 A_1$$

$$r_2 = \bar{B}_1 (\bar{B}_0 A_1 + B_0 A_0)$$

$$r_2 \begin{array}{c|cc} B_1 & B_0 & \\ \hline 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 1 & 0 \\ 1 & 0 & 0 \end{array} \quad a_1: \bar{B}_1 \bar{B}_0 A_2$$

$$a_2: B_1 \bar{B}_0 A_1$$

$$r_2 = \bar{B}_1 (\bar{B}_0 A_2 + B_0 A_1) + B_1 \bar{B}_0 A_1$$

$$r_3 \begin{array}{c|cc} B_1 & B_0 & \\ \hline 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 1 & 0 \\ 1 & 0 & 0 \end{array} \quad a_1: \bar{B}_1 \bar{B}_0 A_3$$

$$a_2: B_1 \bar{B}_0 A_2$$

$$a_3: B_1 \bar{B}_0 A_1$$

$$r_3 = \bar{B}_1 (\bar{B}_0 A_2 + \bar{B}_0 A_3) + B_1 (\bar{B}_0 A_0 \cdot A_1 \bar{B}_0)$$

Figura 17: Desarrollo de la función Shiftleft, parte 1

Fuente: Elaboración Propia

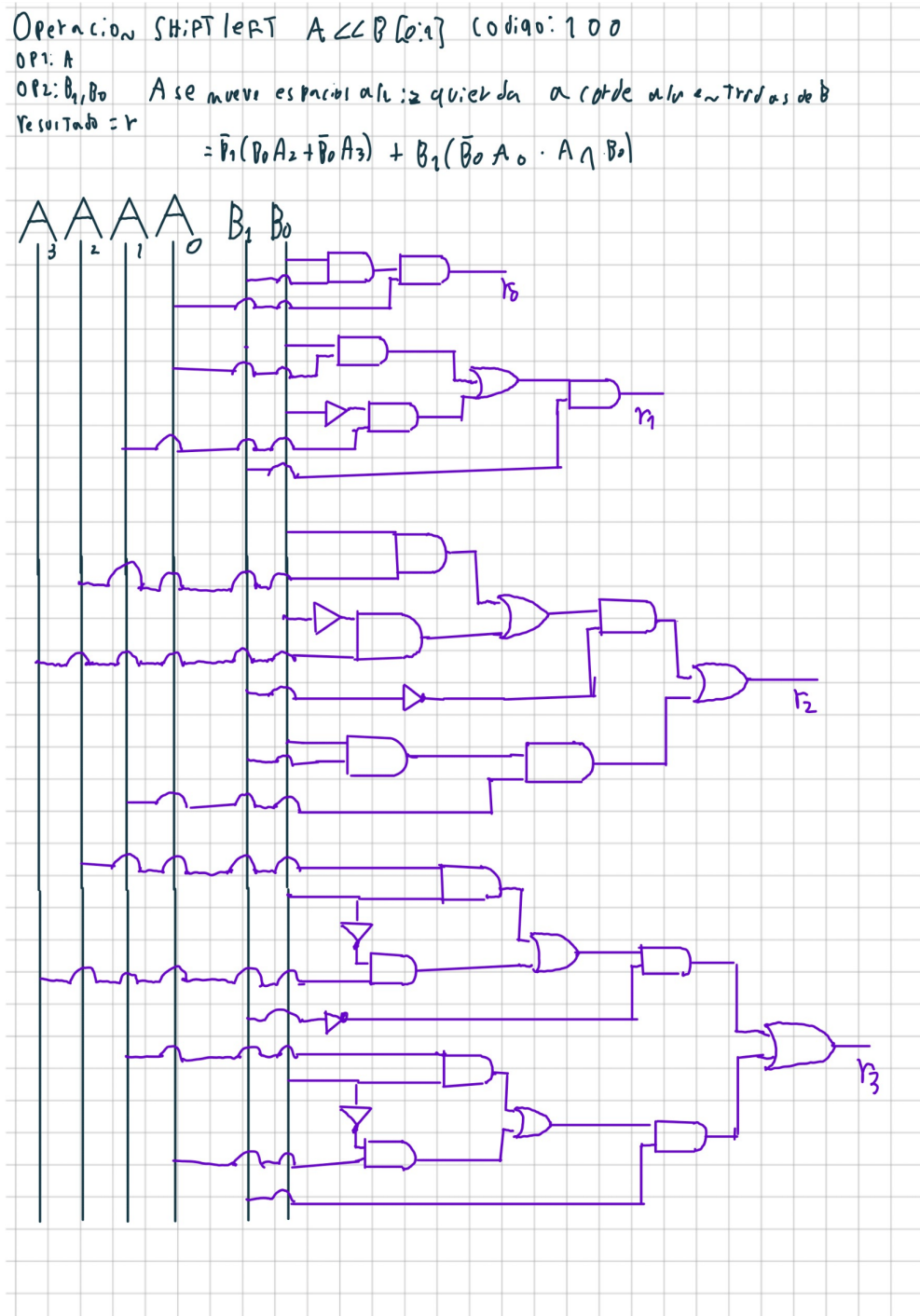


Figura 18: Desarrollo de la función Shiftleft, parte 2

Fuente: Elaboración Propia

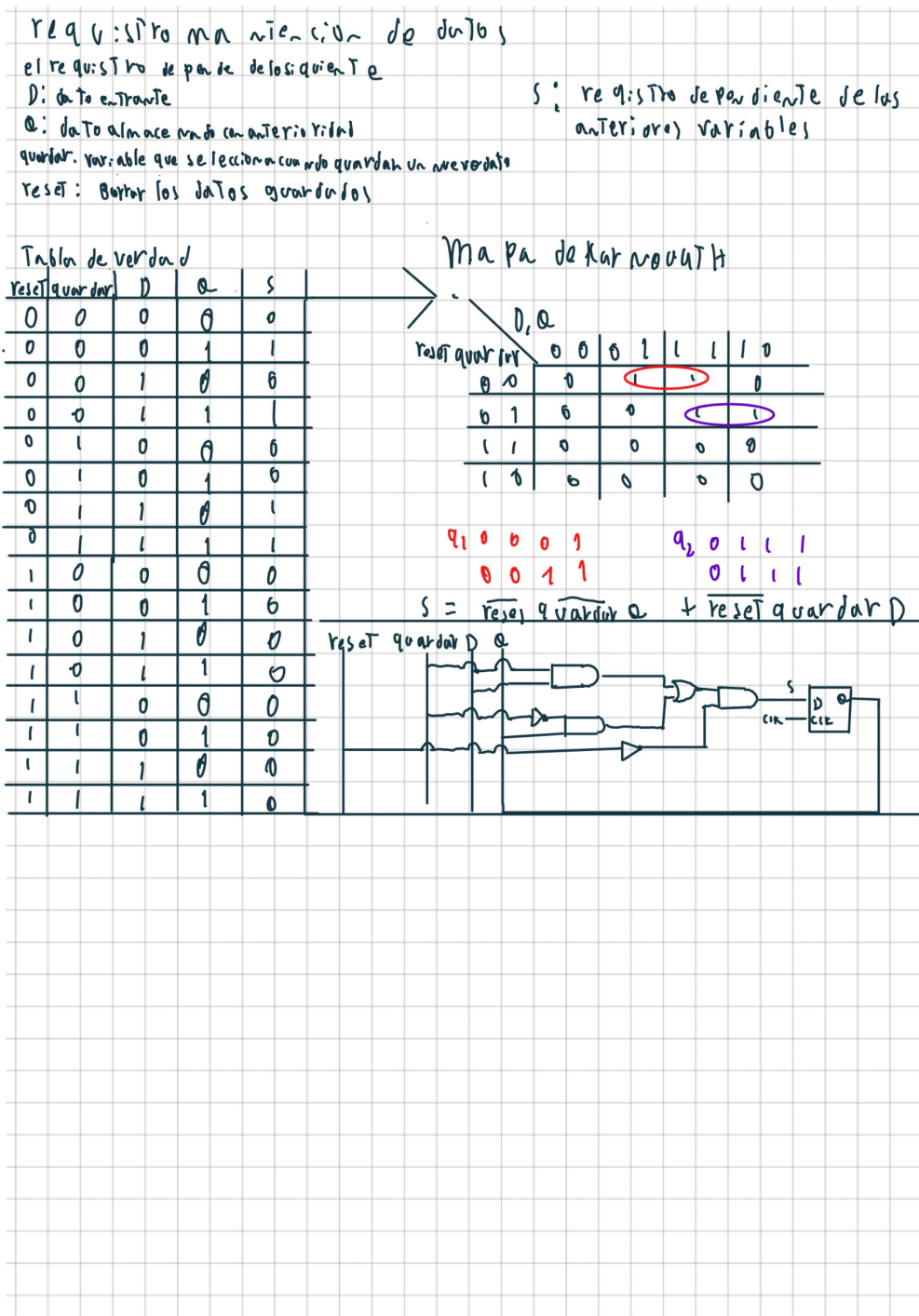


Figura 19: Desarrollo de la función para mantener el registro de datos

Fuente: Elaboración Propia

selector

Selector de 3 bits

Entradas: C_2, C_1, C_0

Salida: O_3, O_2, O_1, O_0

Valores

Suma: S_3, S_2, S_1, S_0 Resta: R_3, R_2, R_1, R_0 (Restas invertidas: R_3, R_2, R_1, R_0)

Left Shift: L_3, L_2, L_1, L_0 Right Shift: R_3, R_2, R_1, R_0

C_2	C_1	C_0	O_3	O_2	O_1	O_0
0	0	0	0	0	0	0
0	0	1	S_3	S_2	S_1	S_0
0	1	0	R_3	R_2	R_1	R_0
0	1	1	R_3	R_2	R_1	R_0
1	0	0	S_3	S_2	S_1	S_0
1	0	1	S_3	S_2	S_1	S_0
1	1	0	X	X	X	X
1	1	1	X	X	X	X

Mapa de Karnaugh

O_i

$C_2 \backslash C_1 C_0$	00	01	11	10
0	0	S	R	R
1	SL	SR	X	X

$$O_i = C_2 C_0 (\bar{C}_1 + C_1 R_i) + C_2 \bar{C}_1 (\bar{C}_0 S_L + C_0 S_R) + \bar{C}_2 C_1 \bar{C}_0 R$$

$$O_i = C_2 (C_1 \bar{C}_0 R + C_0 (\bar{C}_1 S + C_1 R_i)) + \bar{C}_2 C_1 (\bar{C}_0 S_L + C_0 S_R)$$

Figura 20: Desarrollo del selector de 3 bits

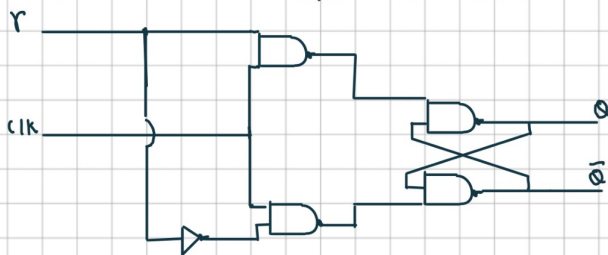
Fuente: Elaboración Propia

Partimos con que necesitamos almacenar los Bits de nuestra salida, para eso, utilizamos latch-D que tienen las siguiente LUT

Latch Tipo D

		E	D	Q	$\bar{Q}$
0	0	1	0	0	1
0	1	1	1	1	0
1	0	0	X	no se maneja	no se maneja

La arquitectura del latch-D, para la salida deseada r



Selector de operador

Entradas: B

r_1 (salida de estado anterior)

P selector

Salida: C

Mapa de Karnaugh

P	C	
0	0	
1	1	

$$C = B\bar{P} + r_1 P$$

Almacenamiento estado de salida anterior

Entrada: a

Salida: r_1

almacenamos la salida de la calculadora y conservamos el estado con un pulso de clock derivado

para eso conectamos 2 latch-D en serie, si quisiese mos almacenar

por mas tiempo deberiamos colocar mas latch-D en serie



Figura 21: Desarrollo del latch y concepto de guardado

Fuente: Elaboración Propia

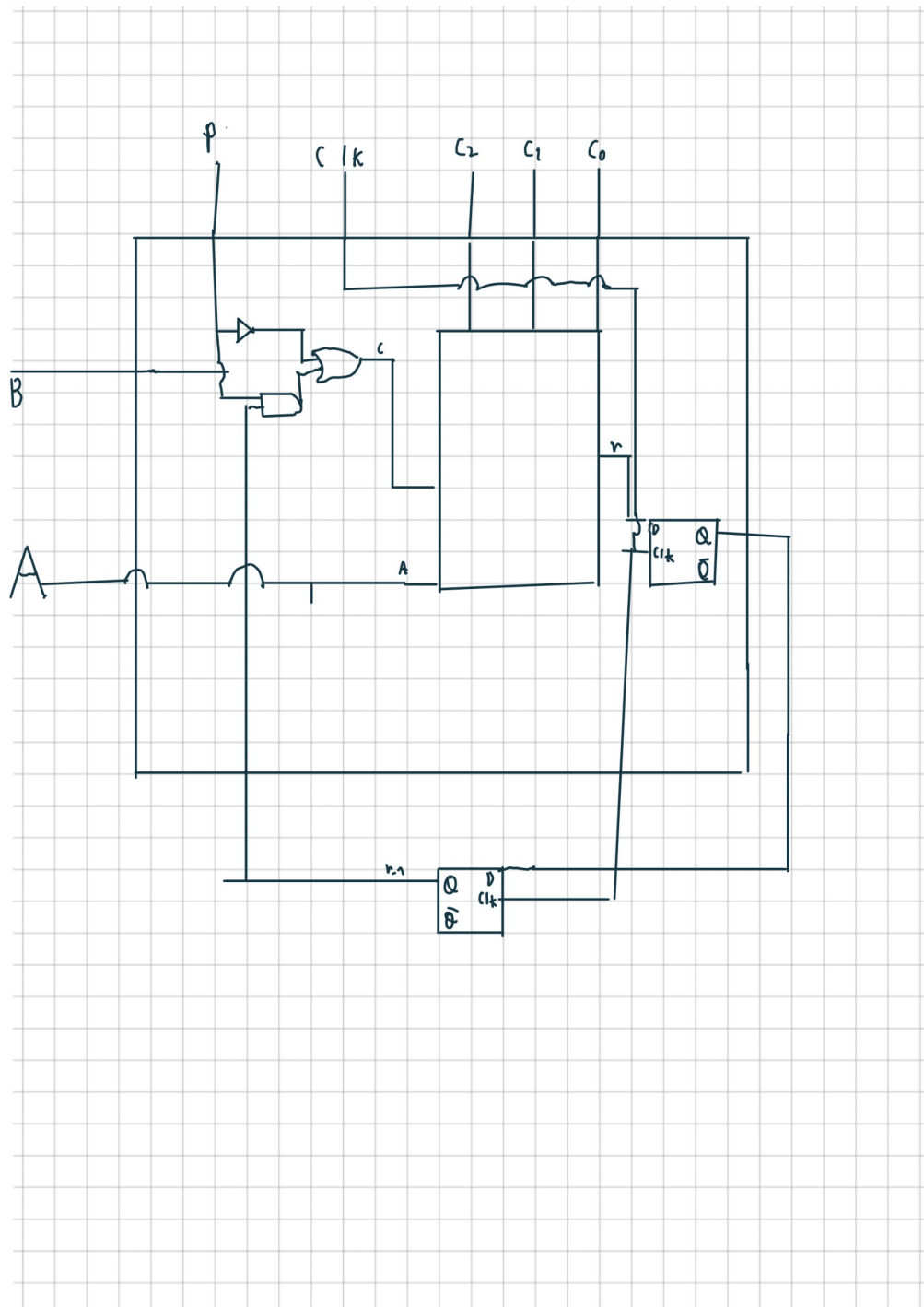


Figura 22: Diagrama de la calculadora resumido

Fuente: Elaboración Propia

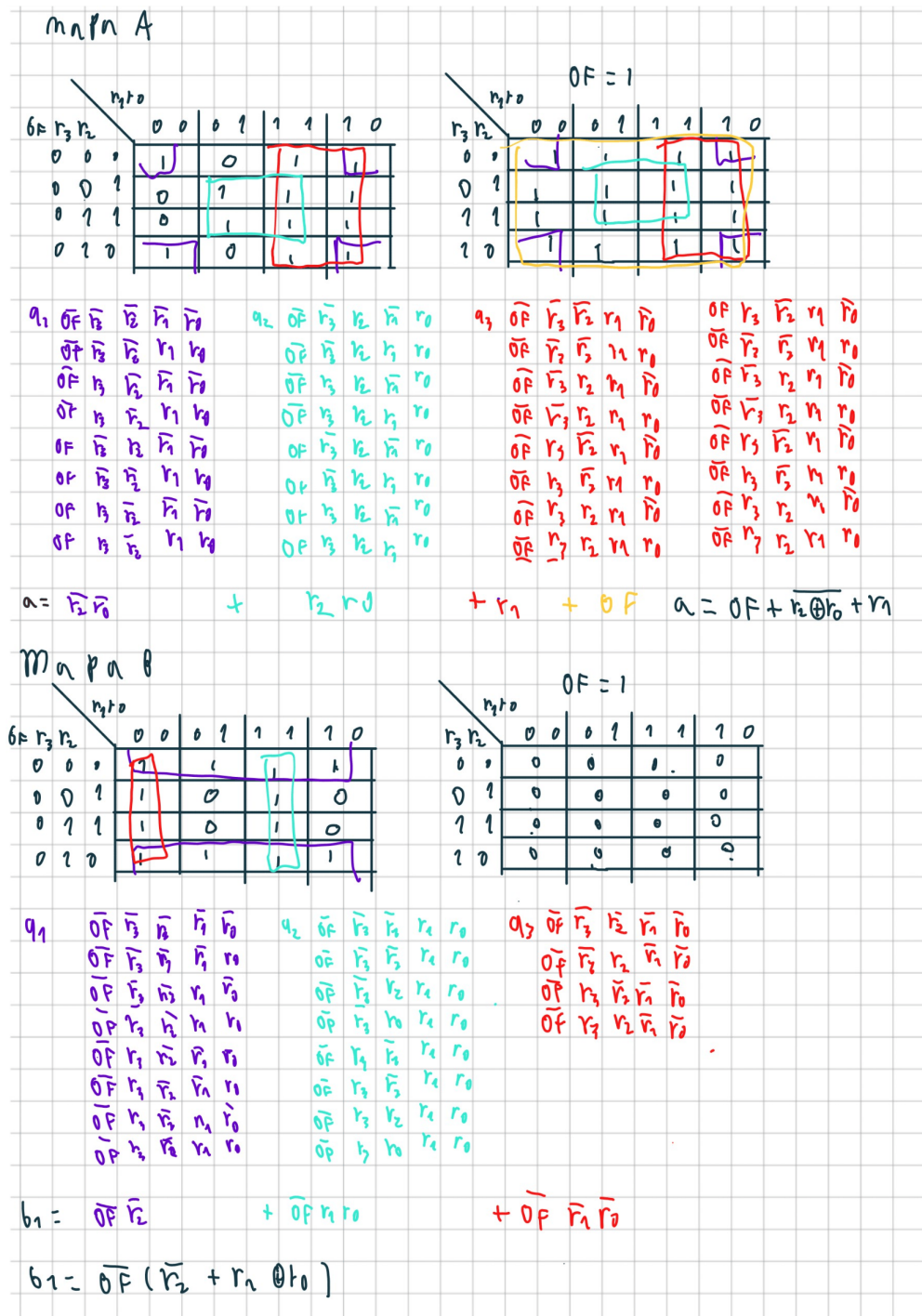


Figura 24: Desarrollo de codificador BCD, Parte 2

Fuente: Elaboración Propia

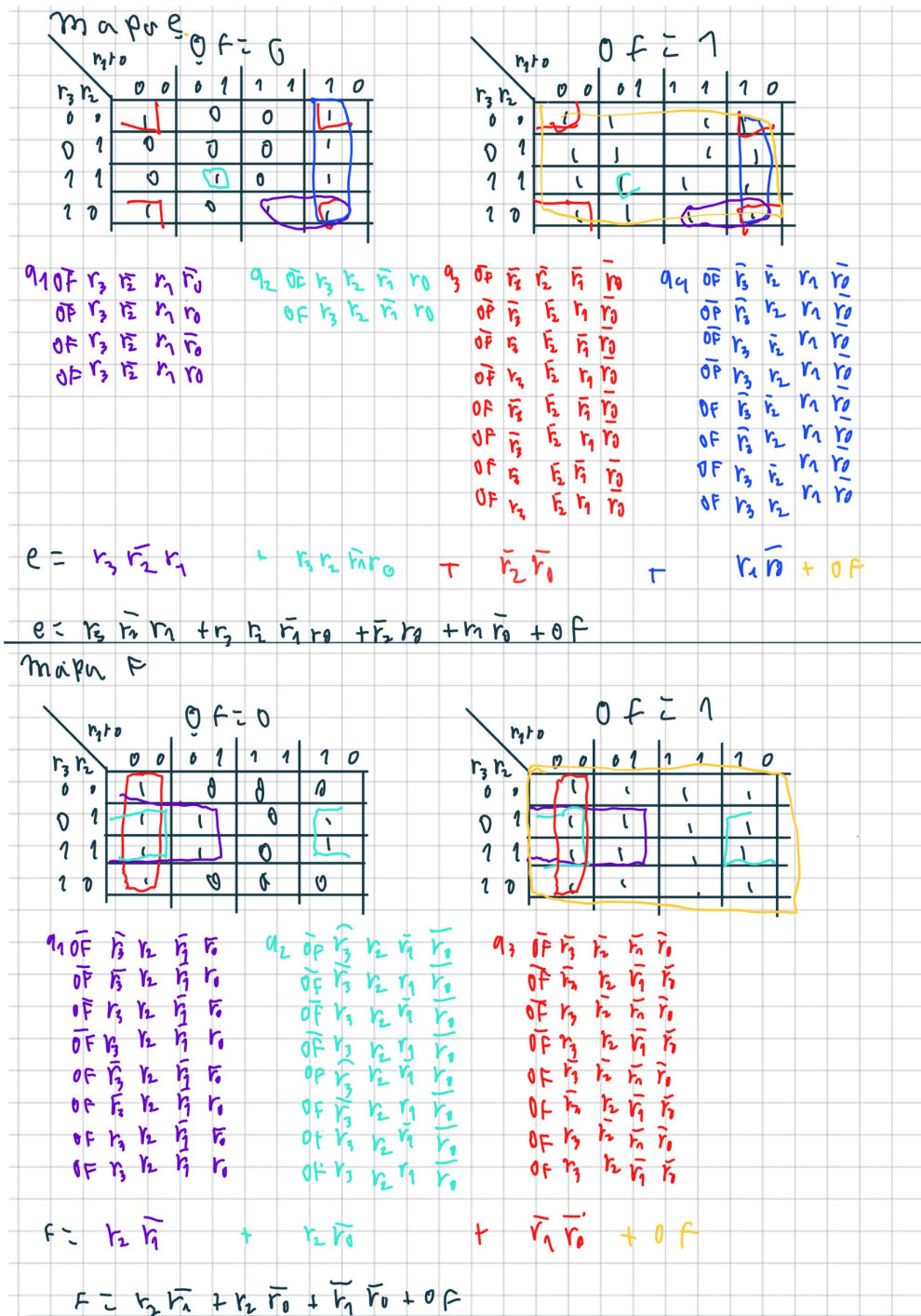


Figura 26: Desarrollo de codificador BCD, Parte 4

Fuente: Elaboración Propia

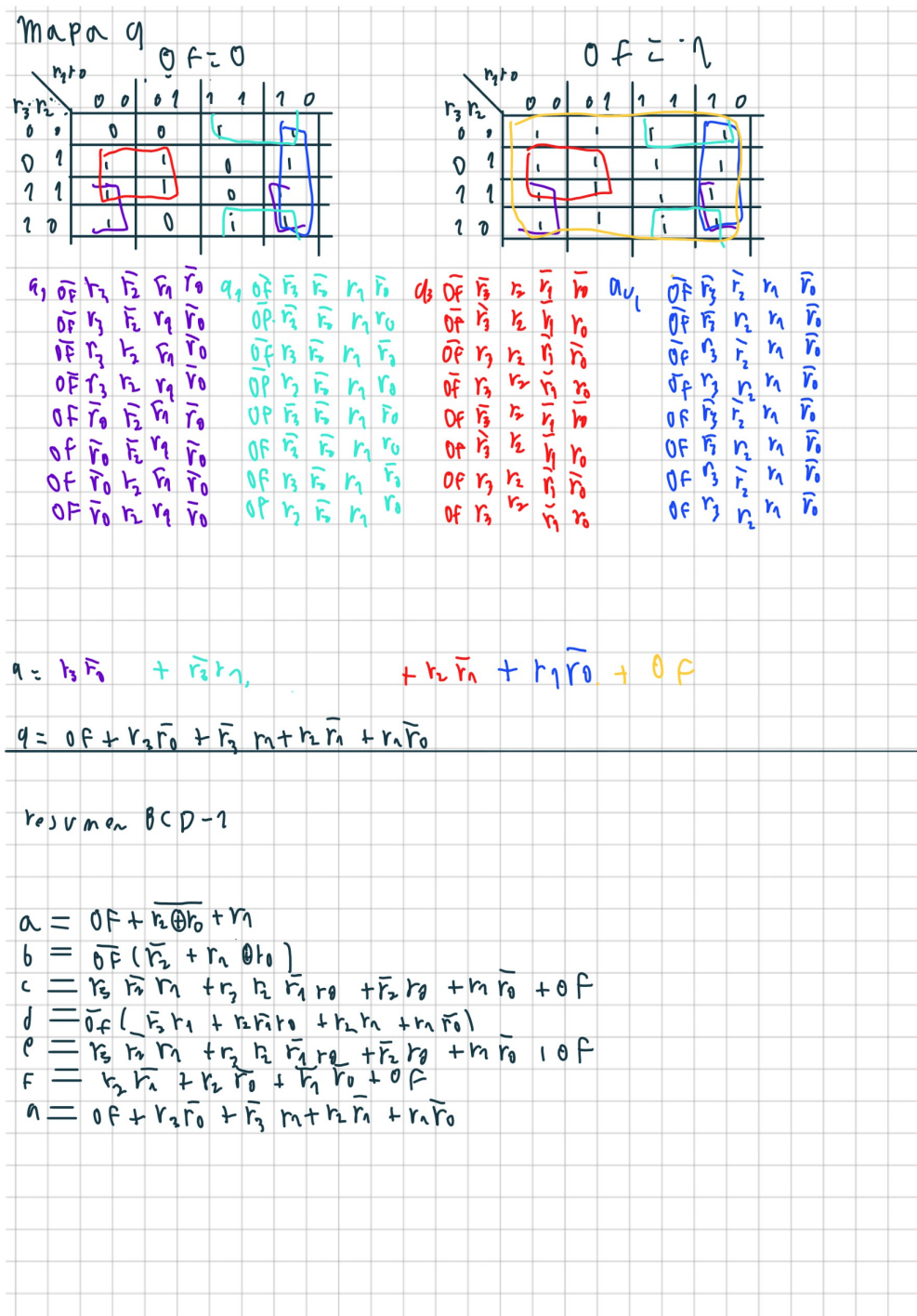
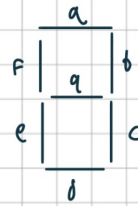


Figura 27: Desarrollo de codificador BCD, Parte 5

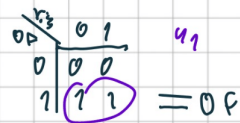
Fuente: Elaboración Propia

mapa de karnaugh BCD-2

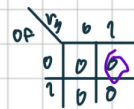
y_3	a	b	c	d	e	f	q
0 0	0	0	0	0	0	0	0
0 1	0	0	0	0	0	0	1
1 1	1	1	1	1	1	1	0
1 0	1	1	1	1	1	1	0



notamos que los mapas de Karnaugh de todos menos q es el mismo, definido por el siguiente mapa



notamos que la variable q se ve solo en la forma



$$\overline{q} \cdot 1 = 0$$

arquitectura



Resumen

$$a = \overline{q} \cdot y_1$$

$$y_{elto} = 0$$

Figura 28: Desarrollo de codificador BCD, Parte 6

Fuente: Elaboración Propia